



Universidad Autónoma de Nayarit

Unidad Académica De Economía

Sistemas Computacionales

“Semana 12 — Tablas Hash: Conceptos y Uso”

Jorge Armando Navarrete Ruiz

Docente.- DR. Eligardo Cruz Sanchez

U. A. ESTRUCTURA DE BASES DE DATOS

Fase 1: COMPRENDE	3
COMPRENDE-1: ¿Qué hace una función hash?	3
# Una función hash toma una clave y la transforma en un número (código hash) que permite ubicar rápidamente	3
# dónde guardar o buscar un par clave→valor dentro de una tabla hash.	3
# COMPRENDE-2: ¿Qué es el factor de carga y por qué importa?	3
# El factor de carga es $\alpha = n/m$ (elementos/buckets). Importa porque si α crece hay más colisiones y el rendimiento	3
# puede degradarse; por eso Python redimensiona (rehash) el dict para mantener $O(1)$ promedio.	3
# COMPRENDE-3: ¿En qué caso el dict de Python deja de ser suficiente?	3
# Cuando necesitas consultas por rango u orden lógico (ej. "puntaje entre 7.8 y 8.5 ordenado"), porque el dict	3
# busca por clave exacta; para rangos/orden dinámico conviene un Árbol Binario de Búsqueda (BST) (S13).	3
# IA-REFLEXION-C: ¿Qué aprendió el equipo que no sabía antes de usar la IA?	3
# Aprendimos que las colisiones son normales (no un error) y que el dict las resuelve internamente. También que	3
# <code>d["k"]</code> lanza <code>KeyError</code> si no existe la clave, mientras que <code>d.get("k")</code> devuelve <code>None</code> (o un default), lo cual es más seguro.	3
Fase 2: APLICA	3
Patrón 1 — <code>buscar_pelicula()</code> :	4
# DECISIÓN: Usamos <code>.get()</code> porque retorna <code>None</code> si la clave no existe y evita <code>KeyError</code> ; así el lookup es seguro.	4
# Patrón 1 — <code>agregar_pelicula()</code> :	4
# DECISIÓN: Insertar/actualizar en dict es $O(1)$ promedio porque el hash lleva al slot; en lista ordenada insertar	4
# puede requerir desplazar muchos elementos ($O(n)$) para mantener el orden.	4
# Patrón 2 — <code>contar_por_genero()</code> :	4
# DECISIÓN: Usamos <code>conteos.get(genero, 0) + 1</code> porque evita <code>if/else</code> , evita <code>KeyError</code> y deja el conteo en una línea clara.	4
# Patrón 3 — <code>agrupar_por_rango()</code> :	4
# DECISIÓN: Inicializamos grupos con listas vacías para asegurar que existan antes de hacer <code>append</code> .	4
# Patrón 3 — <code>peliculas_en_rango()</code> :	4
# DECISIÓN: Usamos <code>grupos.get(rango, [])</code> para que un rango inexistente regrese <code>[]</code> sin <code>KeyError</code> .	4
# Patrón 4A — <code>procesar_lote()</code> :	4
# DECISIÓN: Usamos un set para <code>titulos_existentes</code> porque "título in set" es $O(1)$ promedio; así deduplicamos eficiente.	4
# Patrón 4B — <code>obtener_recomendaciones()</code> :	4
# DECISIÓN: <code>user_id</code> es clave ideal porque el caché se define por usuario: mismo usuario → mismas recomendaciones ya calculadas.	4
Fase 3: REFLEXIONA	4
REFLEXIONA-1: ¿Qué explica la diferencia de tiempos observada?	4
# Lista+bisect es $O(\log n)$ por búsqueda; dict es $O(1)$ promedio por hashing. Por eso el	

dict suele ser más rápido.	4
# REFLEXIONA-2: ¿Por qué el dict mantiene esa ventaja conforme n crece?	4
# Porque el dict depende poco de n ($O(1)$ promedio), mientras que bisect crece como $O(\log n)$. A mayor n, mayor $\log n$.	4
# REFLEXIONA-3: ¿En qué situación el pipeline S11 sería PREFERIBLE al dict?	5
# Cuando necesitas orden o consultas por rango/recorridos ordenados; el dict no es eficiente para rangos.	5
# IA-REFLEXION-R:	5
# La IA nos ayudó a corregir la explicación: no es “binaria más lenta”, sino diferencia de complejidad ($\log n$ vs $O(1)$ promedio)	5
# y el límite del dict para rangos/orden.	5
Fase 4: VALIDA	5
T1–T6: PASARON. Al inicio falló T1 por KeyError al usar <code>d[k]</code> con clave inexistente; se corrigió usando <code>.get()</code> .	5
# También se revisaron fronteras: 8.0 va a “destacado” y 9.0 va a “premium”.	5
# IA-REFLEXION-V:	5
# El error principal fue elegir mal la API (<code>d[k]</code> vs <code>get</code>) para claves que pueden no existir; se corrigió y se validó con T1.	5
# También cuidamos condiciones de frontera en rangos para que los casos T3–T4 pasaran.	5

Fase 1: COMPRENDE

COMPRENDE-1: ¿Qué hace una función hash?

Una función hash toma una clave y la transforma en un número (código hash) que permite ubicar rápidamente

dónde guardar o buscar un par clave→valor dentro de una tabla hash.

COMPRENDE-2: ¿Qué es el factor de carga y por qué importa?

El factor de carga es $\alpha = n/m$ (elementos/buckets). Importa porque si α crece hay más colisiones y el rendimiento

puede degradarse; por eso Python redimensiona (rehash) el dict para mantener $O(1)$ promedio.

COMPRENDE-3: ¿En qué caso el dict de Python deja de ser suficiente?

Cuando necesitas consultas por rango u orden lógico (ej. "puntaje entre 7.8 y 8.5 ordenado"), porque el dict

busca por clave exacta; para rangos/orden dinámico conviene un Árbol Binario de Búsqueda (BST) (S13).

IA-REFLEXION-C: ¿Qué aprendió el equipo que no sabía antes de usar la IA?

Aprendimos que las colisiones son normales (no un error) y que el dict las resuelve internamente. También que

`d["k"]` lanza `KeyError` si no existe la clave, mientras que `d.get("k")` devuelve `None` (o un default), lo cual es más seguro.

Fase 2: APLICA

Patrón 1 — buscar_pelicula():

DECISIÓN: Usamos `.get()` porque retorna `None` si la clave no existe y evita `KeyError`; así el lookup es seguro.

Patrón 1 — agregar_pelicula():

DECISIÓN: Insertar/actualizar en dict es $O(1)$ promedio porque el hash lleva al slot; en lista ordenada insertar

puede requerir desplazar muchos elementos ($O(n)$) para mantener el orden.

Patrón 2 — contar_por_genero():

DECISIÓN: Usamos `conteos.get(genero, 0) + 1` porque evita `if/else`, evita `KeyError` y deja el conteo en una línea clara.

Patrón 3 — agrupar_por_rango():

DECISIÓN: Inicializamos grupos con listas vacías para asegurar que existan antes de hacer `append`.

Patrón 3 — peliculas_en_rango():

DECISIÓN: Usamos `grupos.get(rango, [])` para que un rango inexistente regrese `[]` sin `KeyError`.

Patrón 4A — procesar_lote():

DECISIÓN: Usamos un set para `titulos_existentes` porque "titulo in set" es $O(1)$ promedio; así deduplicamos eficiente.

Patrón 4B — obtener_recomendaciones():

DECISIÓN: `user_id` es clave ideal porque el caché se define por usuario: mismo usuario → mismas recomendaciones ya calculadas.

Fase 3: REFLEXIONA

REFLEXIONA-1: ¿Qué explica la diferencia de tiempos observada?

Lista+bisect es $O(\log n)$ por búsqueda; dict es $O(1)$ promedio por hashing. Por eso el dict suele ser más rápido.

REFLEXIONA-2: ¿Por qué el dict mantiene esa ventaja conforme n crece?

Porque el dict depende poco de n ($O(1)$ promedio), mientras que bisect crece como $O(\log n)$. A mayor n , mayor $\log n$.

REFLEXIONA-3: ¿En qué situación el pipeline S11 sería PREFERIBLE al dict?

Cuando necesitas orden o consultas por rango/recorridos ordenados; el dict no es eficiente para rangos.

IA-REFLEXION-R:

La IA nos ayudó a corregir la explicación: no es "binaria más lenta", sino diferencia de complejidad ($\log n$ vs $O(1)$ promedio)

y el límite del dict para rangos/orden.

Fase 4: VALIDA

T1–T6: PASARON. Al inicio falló T1 por KeyError al usar `d[k]` con clave inexistente; se corrigió usando `.get()`.

También se revisaron fronteras: 8.0 va a "destacado" y 9.0 va a "premium".

IA-REFLEXION-V:

El error principal fue elegir mal la API (`d[k]` vs `get`) para claves que pueden no existir; se corrigió y se validó con T1.

También cuidamos condiciones de frontera en rangos para que los casos T3–T4 pasaran.

```

"""
streamMX_hash_s12.py
Semana 12 · Tablas Hash: Conceptos y Uso
Equipo: [NOMBRE DEL EQUIPO]
Integrantes: [LISTA]
Fecha: [FECHA]

# COMPRENDE-1: ¿Qué hace una función hash? (1-2 oraciones)
# una funcion hash toma un valor de entrada y la convierte en un
numero que se usa como indice para guardar o buscar datos en una
tabla hash

# COMPRENDE-2: ¿Qué es el factor de carga y por qué importa?
#es una medida que nos dice que tan llena esta la tabla antes de
que su capacidad aumente automaticamente

# COMPRENDE-3: ¿En qué caso el dict de Python deja de ser
suficiente?
#cuando los costes de memoria o actualizacion se vuelven un
problema

# IA-REFLEXION-C: [Qué aprendió el equipo que no sabía antes de la
IA]
#no sabia en si que era una tabla hash, pero aprendi que una tabla
hash es como un archivador con muchos cajones enumerados mientras
que una lista ordenada que es similar es como un diccionario
"""

import time
import bisect

#

```

```

# DATOS DEL CATÁLOGO STREAMMX
# Formato: "ID" → (titulo, puntaje_float, genero)
#

```

```

catalogo_base = {
    "MX-001": ("Interestelar", 8.2, "Sci-Fi"),
    "MX-002": ("El Laberinto del Fauno", 7.5, "Fantasía"),
    "MX-003": ("Origen", 9.1, "Sci-Fi"),
    "MX-004": ("Dune Parte 1", 6.8, "Sci-Fi"),
    "MX-005": ("Oppenheimer", 8.7, "Drama"),
    "MX-006": ("La Señal", 7.5, "Thriller"),
    "MX-007": ("Everything Everywhere", 9.4, "Comedia"),
    "MX-008": ("Los Elegidos", 6.2, "Drama"),
    "MX-009": ("The Batman", 8.0, "Acción"),
    "MX-010": ("Gravedad", 7.5, "Sci-Fi"),
    "MX-011": ("Parasite", 9.0, "Thriller"),
    "MX-012": ("Midsommar", 7.1, "Terror"),
}

#

# PATRÓN 1 · LOOKUP DIRECTO — Catálogo dinámico (R1)
# Resuelve el Dilema Final de S11: inserción O(1) sin re-ordenar
#

def buscar_pelicula(catalogo: dict, id_pelicula: str):
    """
    Retorna la tupla (titulo, puntaje, genero) si el ID existe.
    Retorna None sin lanzar KeyError si no existe.

    # DECISIÓN: ¿Por qué usar .get() en lugar de d[k] aquí?
    #asi si no existe la clave retorna none en vez de un
keyerror
    """

    # TODO: implementar con una sola línea usando .get()
    return catalogo.get(id_pelicula)

pass

```



```
def agregar_pelicula(catalogo: dict, id_pelicula: str, datos:
tuple) -> None:
    """
    Agrega o actualiza una película en el catálogo.
    Si el ID ya existe, actualiza los datos (no duplica).

    # DECISIÓN: ¿Por qué la inserción en un dict es O(1) pero en
una lista ordenada era O(n)?
    #por que calculas el hash de la clave, lo que te lleva
directo al numero en ves de recorrer de uno por uno
    """

    # TODO: una sola línea - asignación directa
    catalogo[id_pelicula] = datos
    pass

#
```

```
# PATRÓN 2 · CONTEO DE FRECUENCIAS - Estadísticas por género (R2)
#
```

```
def contar_por_genero(catalogo: dict) -> dict:
    """
    Retorna un dict {genero: cantidad_de_peliculas}.
    Ejemplo: {"Sci-Fi": 4, "Drama": 2, "Thriller": 2, ...}

    # DECISIÓN: ¿Por qué usar .get(genero, 0) + 1 en lugar de
verificar con 'in' primero?
    #asi se evita el keyerror
    """

    conteos = {}
    for id_pelicula, datos in catalogo.items():
        titulo, puntaje, genero = datos
        # TODO: incrementar conteos[genero] en 1.
```

```
        # Manejar el caso en que el género no exista aún
        (sin KeyError).
        conteos[genero] = conteos.get(genero, 0) + 1

    return conteos
```

```
#
```

```
# PATRÓN 3 · AGRUPAMIENTO / BUCKET – Películas por rango de
puntaje (R3)
```

```
#
```

```
def agrupar_por_rango(catalogo: dict) -> dict:
    """
    Agrupa películas en tres rangos de puntaje:
        "popular"    → puntaje en [7.0, 8.0)
        "destacado"  → puntaje en [8.0, 9.0)    (nota: ≥8.0 y < 9.0
pertenece aquí)
        "premium"   → puntaje en [9.0, 10.0]

    Retorna: {"popular": [...], "destacado": [...], "premium":
[...]}
    Cada lista contiene tuplas (id, titulo, puntaje).

    # DECISIÓN: ¿Por qué usar.setdefault() o inicializar el dict
antes del loop?
    # así te aseguras que la lista para cada categoría ya exista
antes de intentar hacer append
    """
    grupos = {"popular": [], "destacado": [], "premium": []}
    for id_pelicula, datos in catalogo.items():
        titulo, puntaje, genero = datos
        # TODO: clasificar puntaje en la categoría correcta y
agregar a la lista.
        # Tip: usar if/elif con comparaciones de punto
flotante.
        if 7.0 <= puntaje < 8.0:
```

```

        grupos["popular"].append((id_pelicula, titulo,
puntaje))
        elif 8.0 <= puntaje < 9.0:
            grupos["destacado"].append((id_pelicula, titulo,
puntaje))
        elif 9.0 <= puntaje < 10.0 :
            grupos["premium"].append((id_pelicula, titulo,
puntaje))

    return grupos

def peliculas_en_rango(grupos: dict, rango: str) -> list:
    """
    Retorna la lista de películas en el rango dado.
    Retorna lista vacía si el rango no existe (sin KeyError).

    # DECISIÓN: ¿qué método de dict usas para evitar KeyError
aquí?
    # el return grupos.get(rango, []) lo evita por que si rango no
existe como clave en vez de fallar devuelve el valor por defecto
    """
    # TODO: una sola línea usando .get() con valor por defecto
    return grupos.get(rango, [])

#

```

```

# PATRÓN 4A · DEDUPLICACIÓN — Detectar títulos duplicados (R4)
#

```

```

def procesar_lote(catalogo: dict, lote: list) -> dict:
    """
    Recibe una lista de tuplas (id, titulo, puntaje, genero).
    Inserta en el catálogo solo las que NO estén ya por título.
    """

```

```

Retorna: {"nuevas": int, "duplicadas": int}

# DECISIÓN: ¿Qué estructura usas para rastrear títulos ya
vistos en O(1)?
# (Pista: el set de Python comparte la misma base
que el dict)
#usas un set como el titulos_existentes
"""
    titulos_existentes = {datos[0] for datos in catalogo.values()}
# set de títulos
    resultado = {"nuevas": 0, "duplicadas": 0}
    for id_p, titulo, puntaje, genero in lote:
        # TODO: verificar si titulo está en titulos_existentes.
        # Si no está: insertar en catalogo y en
titulos_existentes, incrementar "nuevas".
        # Si ya está: incrementar "duplicadas".
        if titulo in titulos_existentes:
            resultado["duplicadas"] +=1
        else:
            catalogo[id_p] = (titulo, puntaje, genero)
            titulos_existentes.add(titulo)
            resultado["nuevas"] += 1

    return resultado

```

```
#
```

```
# PATRÓN 4B · CACHÉ / MEMOIZACIÓN – Recomendaciones por usuario
(R5)
```

```
#
```

```
_cache_recomendaciones = {} # dict global – persiste entre
llamadas
```

```
def _generar_recomendaciones(user_id: str, catalogo: dict) ->
list:
```

```

"""
Simula un motor costoso: tarda ~100ms por llamada.
NO modificar esta función.
"""

time.sleep(0.1) # simula latencia del motor
peliculas = list(catalogo.values())
peliculas.sort(key=lambda x: x[1], reverse=True)
return peliculas[:5]

def obtener_recomendaciones(user_id: str, catalogo: dict) -> list:
    """
    Primera llamada: genera recomendaciones (tarda ~100ms) y las
    guarda en caché.
    Llamadas siguientes: retorna desde caché en O(1) – sin llamar
    a _generar_recomendaciones.

    # DECISIÓN: ¿por qué user_id es la clave ideal del caché en
    este caso?
    #por que las recomendaciones estan pensadas para ser por
    usuario
    """
    # TODO: verificar si user_id está en _cache_recomendaciones.
    # Si está: retornar directamente desde caché.
    # Si no está: llamar _generar_recomendaciones(), guardar
    en caché, retornar.
    if user_id in _cache_recomendaciones:
        return _cache_recomendaciones[user_id]

    recomendaciones = _generar_recomendaciones(user_id, catalogo)
    _cache_recomendaciones[user_id] = recomendaciones
    return recomendaciones

#

```

```

# BENCHMARK – Pipeline S11 (lista + bisect) vs dict

```

```
#
```

```
def benchmark_comparativo(n_valores: list = None):  
    """  
    Compara el tiempo de búsqueda entre:  
    - Pipeline S11: lista de IDs ordenados + bisect.bisect_left  
    - Dict S12: acceso directo por clave  
  
    # REFLEXIONA-1: ¿Qué explica la diferencia de tiempos  
observada?  
    que la búsqueda binaria suele ser mas lenta pero estas sirven  
si se necesitan rangos o iterar en orden  
  
    # REFLEXIONA-2: ¿Por qué el dict mantiene esa ventaja conforme  
n crece?  
    #por que este depende muy poco de n  
  
    # REFLEXIONA-3: ¿En qué situación el pipeline S11 sería  
PREFERIBLE al dict?  
    #cuando quieres cosas ordenadas o por rangos  
    """  
    if n_valores is None:  
        n_valores = [10_000, 100_000, 1_000_000]  
  
    print(f"{'n':>12} | {'Lista+bisect (ms)':>20} | {'Dict  
(ms)':>15} | {'Speedup':>10}")  
    print("-" * 65)  
  
    for n in n_valores:  
        # Generar datos de prueba  
        claves = [f"ID-{i:07d}" for i in range(n)]  
        valor_dummy = ("Película", 8.0, "Drama")  
  
        # Pipeline S11: lista ordenada + bisect  
        lista_ordenada = sorted(claves)  
        t0 = time.perf_counter()  
        for _ in range(1000):  
            objetivo = f"ID-{n//2:07d}"  
            idx = bisect.bisect_left(lista_ordenada, objetivo)
```

```

        _ = lista_ordenada[idx] if idx < len(lista_ordenada)
else None

    t_lista = (time.perf_counter() - t0) * 1000 # ms para
1000 búsquedas

    # Dict S12: acceso directo
    d = {k: valor_dummy for k in claves}
    t0 = time.perf_counter()
    for _ in range(1000):
        _ = d.get(f"ID-{n//2:07d}")
    t_dict = (time.perf_counter() - t0) * 1000

    speedup = t_lista / t_dict if t_dict > 0 else float('inf')
    print(f"{n:>12,} | {t_lista:>20.3f} | {t_dict:>15.3f} |
{speedup:>10.1f}x")

#

```

```

# BLOQUE PRINCIPAL - Casos de demostración y benchmark
#

```

```

if __name__ == "__main__":
    import copy
    cat = copy.deepcopy(catalogo_base)

    print("=" * 60)
    print("PATRÓN 1 · Lookup Directo")
    print("=" * 60)
    print("MX-007:", buscar_pelicula(cat, "MX-007")) # Debe
imprimir tupla
    print("MX-999:", buscar_pelicula(cat, "MX-999")) # Debe
imprimir None
    agregar_pelicula(cat, "MX-013", ("Dune Parte 2", 8.5,
"Sci-Fi"))
    print("MX-013 (nueva):", buscar_pelicula(cat, "MX-013"))

    print("\n" + "=" * 60)

```

```

print("PATRÓN 2 · Conteo por Género")
print("=" * 60)
conteos = contar_por_genero(catalogo_base)
print("Conteos:", conteos)
print("Sci-Fi:", conteos.get("Sci-Fi"), "→ esperado: 4")
print("Drama:", conteos.get("Drama"), "→ esperado: 2")

print("\n" + "=" * 60)
print("PATRÓN 3 · Agrupamiento por Rango")
print("=" * 60)
grupos = agrupar_por_rango(catalogo_base)
for rango, peliculas in grupos.items():
    print(f"    {rango}: {len(peliculas)} películas")
print("Premium (9-10):", len(peliculas_en_rango(grupos,
"premium")),
    "→ esperado: 3")
print("Rango inexistente:", peliculas_en_rango(grupos,
"epico"),
    "→ esperado: [])")

print("\n" + "=" * 60)
print("PATRÓN 4A · Deduplicación")
print("=" * 60)
lote_con_duplicados = [
    ("MX-013", "Interestelar", 8.2, "Sci-Fi"),      # duplicado
por título
    ("MX-014", "Avatar: El Camino del Agua", 7.6, "Acción"),
# nueva
    ("MX-015", "Parasite", 9.0, "Thriller"),      #
duplicado por título
    ("MX-016", "Tár", 7.8, "Drama"),              # nueva
]
resultado = procesar_lote(copy.deepcopy(catalogo_base),
lote_con_duplicados)
print("Resultado:", resultado, "→ esperado: {'nuevas': 2,
'duplicadas': 2}")

print("\n" + "=" * 60)
print("PATRÓN 4B · Caché de Recomendaciones")
print("=" * 60)
t0 = time.perf_counter()

```



```

r1 = obtener_recomendaciones("user_42", catalogo_base)
t1 = time.perf_counter() - t0
t0 = time.perf_counter()
r2 = obtener_recomendaciones("user_42", catalogo_base)
t2 = time.perf_counter() - t0
print(f"1er llamado: {t1*1000:.1f}ms")
print(f"2do llamado: {t2*1000:.1f}ms")
if t1 > 0 and t2 > 0:
    print(f"Speedup caché: {t1/t2:.0f}× → esperado: ≥ 100×")

print("\n" + "=" * 60)
print("BENCHMARK – Pipeline S11 vs Dict S12")
print("=" * 60)
benchmark_comparativo()

# — Evidencia embebida al final del archivo

```

```

# # ESTRATEGIA: [El equipo describe en 2-3 oraciones su
enfoco general
# #           para los cuatro patrones – qué decidieron y
por qué]

# # VALIDA: [El equipo confirma qué casos de prueba pasaron y
cuáles
# #           requirieron ajuste, con explicación del error
encontrado]
#el primer caso de prueba al inicio no paso por que use d[k]
en vez de .get al inicio, saliendo el error del keyerror pero
luego lo pudmos corregir
#el otro caso que fallo fue el segundo pero solo por que me
habia equivocado de simbolo

# # IA-REFLEXION-A: [Qué aprendió el equipo durante APLICA al
usar la IA]
#aprendi a como implementar de forma adecuada las tablas hash
y tambien la diferencia entre d[k] y .get entre otras cosas

# # IA-REFLEXION-R: [Se completa en REFLEXIONA]
# principalmente la ia nos corrigio la opcion de leer sin error
cuando la clave no existe usando el dict.get()

```

```
# # IA-REFLEXION-V: [Se completa en VALIDA]
# # IA-REFLEXION-P: [Se completa en PROFUNDIZA]
#anticipo que seguiremos con los arboles por que es una
estructura relativamente importante que no hemos llegado a ver
```